Stolpe,
Graal-LSP

Graal-LSP: A Zero-overhead Polyglot Language Server Implementation for the GraalVM

Graal-LSP: Eine Language Server Implementierung für die GraalVM ohne Beinträchtigung der Ausführungsgeschwindigkeit

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

December 26, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Contributions . . . 2
1.2. Outline . . . 3

2. Context **5**
2.1. Source Code Editors . . . 5
2.2. Language Server Protocol . . . 6
2.3. Terms and Concepts . . . 8
2.4. Language Implementation Frameworks . . . 9

3. Approach **11**
3.1. Completion of Globals and Locals . . . 11
3.2. Completion of Locals Using Run-time Information . . . 12
3.3. Completion of Properties . . . 15
3.4. Transferring the Approach to Other Requests . . . 15
3.5. Language Server Initialization . . . 17
3.6. Parsing Source Code . . . 18
3.7. Diagnostics . . . 18
3.8. Mapping Caret Position to AST Node . . . 18
3.9. Static Completion of Globals and Locals . . . 19
3.10. Completion of Locals Using Run-time Information . . . 21
3.11. Completion of Object Properties . . . 24
3.12. Signature Help . . . 26
3.13. Hover Information . . . 26
3.14. Document Symbols . . . 27
3.15. Workspace Symbols . . . 28
3.16. Document Highlight . . . 28
3.17. Goto-definition . . . 28
3.18. Finding All References . . . 29
3.19. Triggering Run-time Information Collection . . . 29

4. Implementation **31**
4.1. Architectural challenges . . . 31
4.2. Language Server Initialization . . . 31
4.3. Parsing Source Code . . . 31
4.4. Diagnostics . . . 32
4.5. Mapping Caret Position to Abstract Syntax Tree (AST) Node . . . 32
4.6. Static Completion of Globals and Locals . . . 32
4.7. Completion of Locals Using Run-time Information . . . 32

4.8. Completion of Object Properties . . . 33
4.9. Signature Help . . . 33
4.10. Hover Information . . . 33
4.11. Document Symbols . . . 34
4.12. Workspace Symbols . . . 34
4.13. Document Highlight . . . 34
4.14. Goto-definition . . . 34
4.15. Finding All References . . . 34
4.16. Triggering Run-Time Information Collection . . . 34
4.16.1. LSP client - VSCode extension example . . . 35
4.17. Requirements on Truffle Language Implementations . . . 35

5. Experience Report **37**
5.1. Limitations of Graal-LSP . . . 37
5.2. Limitations of Our Approach . . . 38

6. Related Work **39**
6.1. Reusing existing tools . . . 39
6.2. Language Workbenches . . . 39
6.3. Run-time type collection . . . 40
6.4. Language-agnostic tagging of symbols . . . 40

7. Conclusion and Future Work **41**
7.1. Future Work . . . 41

A. First Unimportant stuff. **47**

List of Figures

2.1. An exemplary communication of language client and server including initialization, opening and changing a file, and requesting a code completion and goto-definition. 7
2.2. Truffle languages and tools hosted by the Truffle framework on top of the GraalVM 10

3.1. lang specific server vs lang-agnostic server 16
3.2. An existing Visual Studio Code extension for the Python language showing the completions of globals and locals available in a function scope, together with reserved keywords. 20
3.3. A Visual Studio Code extension for the Python language showing all detected symbols in the current document. There are different icons for modules, classes, functions and variables. 27

List of Tables

3.1. Selected Language Server Protocol (LSP) features and their descriptions 17

List of Listings

1. An example Python code snippet showing a function with two parameters and two local variables to explain completion of globals and locals. 11
2. A Python code snippet showing a property completion example. . . 15
3. An example Python code snippet showing two functions to explain scopes. 20
4. A Python code snippet showing an object property completion example. 25
5. A Python code snippet example to illustrate migrating cached runtime information. 30

List of Abbreviations

LSP	Language Server Protocol
IDE	Integrated Development Environment
AST	Abstract Syntax Tree
API	Application Programming Interface
VM	Virtual Machine
URI	Uniform Resource Identifier
VSCode	Visual Studio Code
TDD	Test-Driven Development
DSL	Domain Specific Language
IR	Intermediate Representation
IR	Intermediate Representation

1. Introduction

- language development is challenging task
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- who provides the tools? language developers vs. language users
- normally tools are developed for a specific language and cannot be reused or need lots of adaptation
- In the Truffle framework, tooling (instrumentation) is part of the framework and therefore tools are implemented language-agnostic once
- Truffle has already some language-agnostic tools implemented (debugger, profiler, ...) and new ones can be easily built
- Truffle tooling support does not interfere with implementing high-performance languages
- Compared to a full IDE, some tools are missing
- Implementing LSP can provide further IDE-like features
- We propose an approach to build a generic LSP server on top of Truffle with:
  - minimal to zero language runtime performance overhead
  - minimal API changes to Truffle
  - minimal impl and maintenance overhead for language developers
- Benefits for the language developers: get IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients + consistent dev-tools experience for different Truffle languages
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM
- We have not implemented all LSP features, but started with the ones which are more easily implementable with Truffle or have are considered as important (code completion)
- ?: focus on dynamically type languages? Or is this focus already included in Truffle (most impls are dynamically typed langs)?

Alternative Introduction

- language development is challenging task
- but we need new languages (or new implementations of old langs) to deal with todays challenges of modern software systems (not only DSLs)
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- to ease the development process, language implementation frameworks exist
- but these frameworks either focus on run-time performance (Truffle, RPython) or rich tooling support (language workbenches)
- second: if tooling support is present, it is limited to one or two front-ends (e.g. an Eclipse- or IntelliJ-plugin) the end-users are forced to use -> pain point if programmers need to not only learn new lang, but also new IDE semantics

explain deeper in extra sub-section?

- we want both
- with new concepts like LSP we have to think about, if we can build a language-agnostic LSP for run-time focused lang impl frameworks
- this would add further IDE-like features out of the box for langs build with these frameworks
- challenging is lang-agnostic IDE-like tooling support for dynamic languages (no static types)
- We propose an approach to build a generic, language-agnostic LSP server on top of run-time focused lang impl frameworks with:
  - minimal to zero language runtime performance overhead
  - minimal API changes
  - minimal impl and maintenance overhead for language developers
- Benefits for the language developers: provide IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients => consistent dev-tools experience for different (Truffle)languages
- to do so, we exploit as much static infos of the framework as we can
- where not possible, we try to learn about our program by observing its execution
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM

1.1. Contributions

- Contribution in detail:
  - An approach to build a generic LSP server on top of Truffle
  - An implementation of said approach: Graal-LSP
  - Proposals to extend the Truffle Framework with new interfaces needed to support more LSP features
  - An experience report how well existing Graal/Truffle-implemented languages work with Graal-LSP(with and without implementing the said interface proposals)

1.2. Outline

2. Context

In this chapter we explain terms, concepts and technologies helping to understand our approach in Chapter 3 and the implementation of Graal-LSP in Chapter 4.

2.1. Source Code Editors

A source code editor bundles different features to support developers in their process of writing code. Typically, code editors like vim, Sublime Text, Atom, or Visual Studio Code, focus on editing file-based programming or scripting languages. In addition to the actual text editor view, modern graphical code editors also consist of a file explorer, graphical debugger interface, or build-tools integration, which are often made available as optional extensions for a variety of languages. Popular Integrated Development Environments (IDEs) like Eclipse, Visual Studio, or IntelliJ also have a code editor view as main component, but focus on a richer support of one specific language by providing tailor-made, integrated tools. In the following we will describe selected features of source code editors, which are covered by the LSP described in Section 2.2.

footnote?

footnote?

Navigational Features Source code editors often provide some actions or views to navigate through source code files. Additionally to a pure textual search, there might be a hierarchical outline view of symbols of the current source file or actions to find all definitions or references of a symbol.

Informational Features Another group of features tries to visualization source code related information, e.g. by allowing to hover with the cursor over a function call, to display signature and documentation, or over a variable, to display type information. Furthermore, if the cursor is placed at a symbol in the code, all other locations of the symbol are often highlighted in the current file. For example all read and write accesses of a local variable. If a language parser detects syntax errors or a linter provides some warnings, these information are also displayed in the code editor, typically by underlining the affected source sections.

Code-modifying Features Source code editors often provide some actions to format the whole file or a code snippet according to some formatting rules. Additionally there might be a lightweight refactoring support, like the renaming of a symbol in the current file or all files in the workspace. More sophisticated refactoring tools are rather found in IDE than source code editors.

Code Completion An important feature of source code editors and IDEs is code completion, which is typically integrated seamlessly into the textual editor view. We describe this feature in more detail in the following, because we will use it as running example in our approach in Chapter 3.

Code completion can be triggered manually, e.g. in most editors by pressing `Ctrl+Space`, or automatically when a user types a trigger character. For example in the Java programming language, such a trigger character would be the dot character inserted after a variable. The suggested completion items could therefore be all names of the Java methods or fields declared by the variable's type and super types. If available, a documentation is typically also included in the completion item and for callable objects also a signature. This kind of code completion, where the completion items are depending on the symbol in front of the trigger character, we call in our thesis *completion of properties*. For non-object-oriented languages, *completion of properties* might not be supported or be limited to special language elements, which understand some kind of *property* semantic. *Completion of properties* is often a challenge for languages with untyped symbols, like Python or JavaScript, so that only some IDEs provide a proper code completion system. IDEs like PyCharm typically use static type inference to obtain types for Python expressions.

ref

The other kind of code completion, which you typically get in editors when the caret is placed in an empty line and the user triggers the completion manually, we call *completion of globals and locals*. In this case, the completion items are depending on the surrounding scopes. With *globals* we refer to all the symbols which we can derive from global scopes of languages and with *locals* we analogously refer to all symbols which we can derive from local scopes of languages. In the Python programming language for example, globals are built-in functions or built-in classes. Locals of a function scope are function parameters or local variables. Locals of a class scope are fields or methods. *Completion of globals and locals* might also be triggered automatically, when the user starts typing a symbol, so that matching globals and locals are suggested.

Tim fragen - werden ja nicht bei locals vorgeschlagen

Because code completion is a core feature and is frequently auto-activated while typing, Asaduzzaman has collected requirements of code completion systems [1]. Most of them address the fact, that the completion system should not interrupt the user's flow of code writing.

2.2. Language Server Protocol

In 2016 a group of IT companies, including Microsoft, Red Hat, and Codenvy, started to standardize the Language Server Protocol (LSP) [8]. This protocol describes an approach to tackle the problem of language and tool developers, which need to repeatedly implement language support (e.g. code completion, jump-to-definition and syntax error reporting) for every new code editor, IDE or development tool. The LSP defines a *language server* and a *language client*. The server needs to be implemented only once for every language, providing the language-specific smarts. A development tool implementing a language client can make use of any language server and therefore support multiple languages with minimal effort.

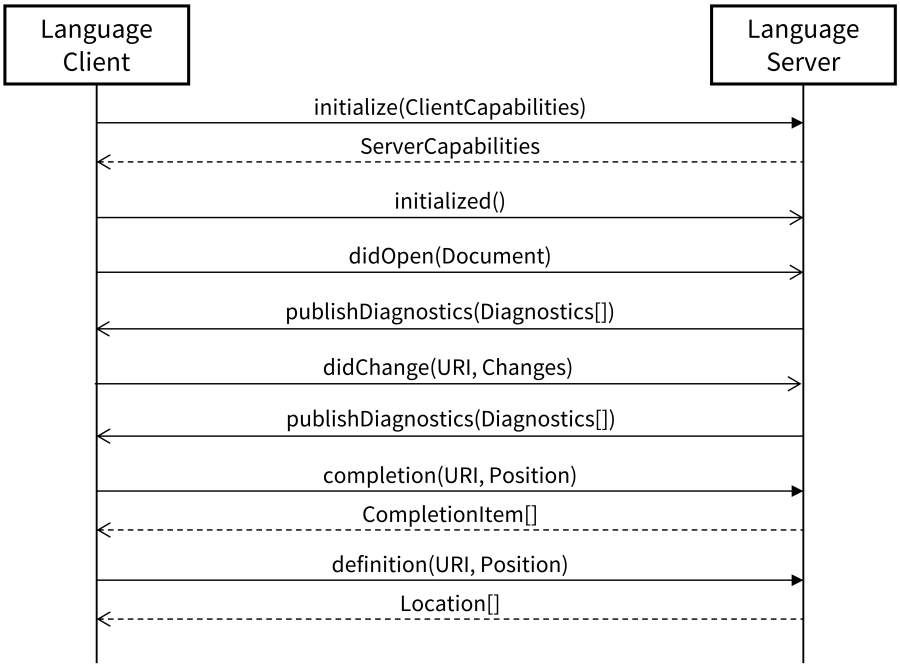


Figure 2.1.: An exemplary communication of language client and server including initialization, opening and changing a file, and requesting a code completion and goto-definition.

At the date of writing, language servers for more than 50 languages exist[1], including almost all popular programming and scripting languages. Not all implementations support the full feature set defined in the LSP. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

The LSP version 3.7.0, which is the reference for our approach and implementation of Graal-LSP, specifies requests for typical features of a source code editor: e.g. code and signature completion, navigational features (goto-definition, find all references and implementations, etc.), information on hover, or code formatting. Syntax highlight support is not covered in the protocol and still needs to be handled by development tool specific configuration files or Application Programming Interfaces (APIs).

Client and Server Communication LSP client and server communicate via JSON-RPC[2] requests, responses and notifications. Source code files are referenced by Uniform Resource Identifiers (URIs). A location in the source code is encoded as line and column tuple. During initialization client and server can exchange which features are supported.

Figure 2.1 shows an exemplary communication between server and client. After initialization, the client notifies the server that a file has been opened. The server

figure: context instead of position, dots between completion and definition, dots at end

[1] https://microsoft.github.io/language-server-protocol/implementors/servers/ Sep. 2018
[2] https://www.jsonrpc.org/specification

parses the source code, detects a syntax error and pushes diagnostics to the client. The user fixes the displayed syntax error and the client sends these changes incrementally to the server. After parsing again on the server side, no errors occur, but the server needs to explicitly notify the client about that, so that the client can clear the currently displaying diagnostics. Then, the client sends a request for code completion at a specific source location, typically the caret position. The server obtains completion items depending on the provided context, which might also contain the trigger character, and sends the items to the client. Later, a request to find one or more definitions for a symbol at a position in the source code is sent to the server. The server responds with a list of locations, if found.

LSP4J Our implementation of Graal-LSP uses LSP4J[3] version 0.4.1 as Java binding for the Language Server Protocol. This framework maps the LSP requests and notifications to methods of Java interfaces so that language server implementors do not have to deal with the JSON-RPC communication layer and can make use of Java's type system.

Is this important?

Visual Studio Code We used Microsoft's source code editor Visual Studio Code (VSCode)[4] to create an exemplary language client extension and experiment with Graal-LSP.

2.3. Terms and Concepts

consume in a "Virtual Execution Environment" section?

Abstract Syntax Tree (AST) Parsers for programming languages usually transform plain text source code into an Intermediate Representation (IR) which can be processed by compilers, interpreters or programming environment tools. An AST is such a typical representation which stores structural information of the source code implicitly in its tree structure. Therefore, concrete syntactic elements needed to write parsable code in plain text can be omitted. In general, inner nodes of the AST represent programming language operations and leaf nodes store the values needed for these operations. Depending on the use case, an AST can store additional information in its nodes. For example the lexical source section from which a node was derived. Programming environment tools can use these information to display any kind of errors, warnings or hints directly in the plain text source code. An AST interpreter evaluates an AST by traversing its tree structure and executing the operations defined in the nodes.

Scopes and Frames A Scope defines a region of a program in which a set of name bindings is valid, e.g. a local variable name. Most programming languages have a concept of a global (program wide) scope and local scopes defined by functions,

[3] https://github.com/eclipse/lsp4j
[4] https://code.visualstudio.com

blocks or modules. During execution of a program, frames store the actual values of the name bindings of visible scopes.

ref Frame: "Advanced Compiler Design and Implementation" book p.114

expression, statement und co?

2.4. Language Implementation Frameworks

Language Implementation Frameworks help a language developer in the process of creating a new language. The feature set varies immensely, depending on the purpose of the framework. Language workbenches, like Spoofax, Xtext, or MPS, are used to develope and compose Domain Specific Languages (DSLs) together with a corresponding IDE with minimal effort. The focus is therefore on the rich support for tools to easily and efficiently create, compose and work with DSLs. Run-time performance is not the main focus. On the other side of the spectrum, there are frameworks like Truffle [17] or RPython, which focus on implementing high-performance programming languages. Tooling for the developed languages and especially IDE-like tooling, which you get by language workbenches, has not been a focus yet. However, Truffle is an exception, due to its smart integration of an API to build tools.

ref to Fowler, Spoofax, Xtext etc - footnote vs ref?

ref

Truffle and GraalVM Truffle [17] is a language implementation framework written in Java. The language developer uses the framework to write an AST interpreter, which is able to optimize itself during interpretation [20]. To achieve that, Truffle uses run-time type information to rewrite AST nodes. Callable elements of a language, e.g. functions or methods, are modeled as root nodes. They can be executed by providing a frame object. A node can have a source section attribute which encodes the range in the source code file it relates to. How source code is parsed and the AST is structured leaves Truffle in the responsibility of the language implementation.

Truffle has built-in support for developing language-agnostic tools by providing an instrumentation API [12]. It is designed to work well with the optimization techniques for language implementations so that instrumentation can be done with nearly zero-overhead [13]. Instruments can be installed by putting them on the Java class path and then be registered and unregistered dynamically as listeners for different events. For example, an instrument can be notified when a new source or source section is loaded, a statement is going to be executed or an expression will return a value. It is also possible to alter the current execution flow and let a function return a different value when called under certain conditions. This enables building powerful tools like debuggers or profilers which need to be implemented only once and can then be used for all Truffle languages. To control which AST nodes of a language are available for instrumentation, language implementors need to mark nodes as instrumentable. A tagging mechanism allows adding meta information, like "this node represents a statement in my language", which can be used by instruments to filter for specific nodes. For example, a debugger can facilitate these information to implement stepping. The tools available at the date of writing are mostly run-time tools like debuggers, dynamic profilers and language extensions [13].

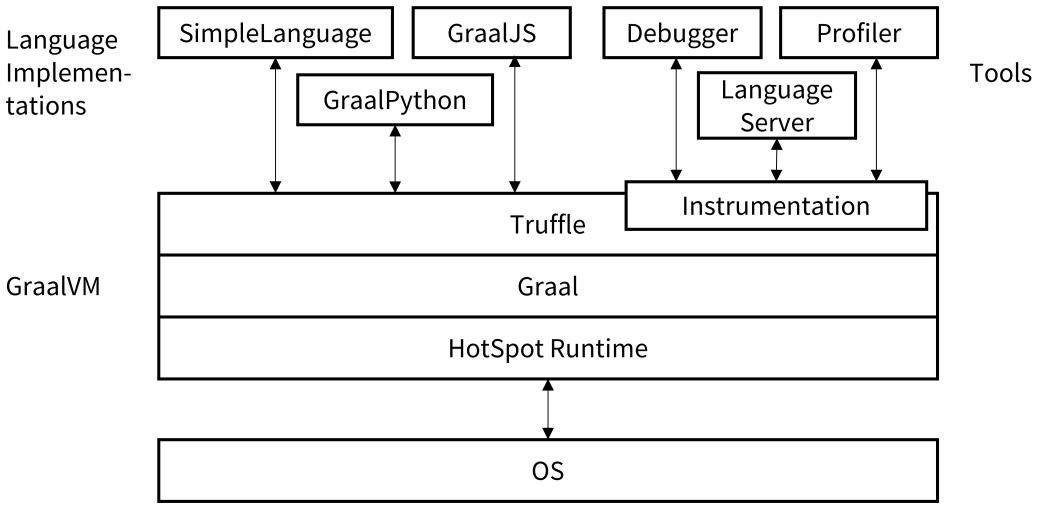


Figure 2.2.: Truffle languages and tools hosted by the Truffle framework on top of the GraalVM

Cross-language interoperability is another feature of the Truffle framework [4]. Language implementations can be hosted by the Truffle framework in a multi-language run-time and communicate via language-independent messages. Foreign language objects can be accessed directly without marshalling.

To get high-performance language implementations, Truffle uses the language-independent just-in-time compiler named Graal. It has its roots in the research project of the Maxine VM [16]. Among other optimizations, Graal applies partial evaluation to compile stable AST subtrees into optimized machine code [18, 3]. These optimizations do not stop at language boundaries, so that the access of foreign language objects can also be optimized.

cite or footnote?

GraalVM[5]is the umbrella term for a HotSpot VM which is bundled with the Truffle language implementation and instrumentation framework and the Graal compiler, building a high-performance polyglot VM for applications written in a Truffle-implemented language [19]. Figure 2.2 visualizes that architecture.

RPython

- bytecode interpreter
- type inference [7]

[5] https://www.graalvm.org/

3. Approach

In Chapter 1 we described the problem, that run-time performance focused language implementation frameworks lack IDE-like tooling support for the create languages. Therefore we suggest to build a language-agnostic language server on top of these kind of frameworks to start closing the tooling gap. Typically a language server is developed for one specific language only and the server implementation can exploit any kind of language-specific information. To implement a language-agnostic language server, we have to build upon the abstract concepts of language implementation frameworks, which apply to all languages developed in these frameworks.

- focus on performance-focused and interpreter-based lang impl frameworks (like Truffle, RPython)
- interpreter-based = the developer writes an interpreter to define the language semantics (bytecode or AST interpreter)

bullet points to text

In the following, we use the example of code completion to described our approach. Fist of all, we show which kind of data is needed to respond to the *completion* request of LSP. Then, we suggest how to obtain the needed information in a language-agnostic way and where the language implementations need to hook into.

3.1. Completion of Globals and Locals

As described in Section 2.1, there are two kinds of code completions we have to distinguish. We start with the Python code snippet in listing 1 to explain how to support code completion of globals and locals.

Listing 1: An example Python code snippet showing a function with two parameters and two local variables to explain completion of globals and locals.

```
1  def foo(param1, param2):
2      x = 42
3
4      y = param1 + param2
5
6      print(x * y)
```

Imagine, the listed Python code snippet is the content of a file which was opened in an LSP-supporting source code editor and the language client implementation has already reported the opening of the file using the *didOpen* notification to our imaginary language server. This notification includes the content and the URI of the file. Now, a *completion* request is sent to the server with parameters `line=3` and `character offset=4`. This means, the completion is requested in the empty line

between the assignments of the two local variables `x` and `y`. We would expect a completion result to include items for all Python globals (e.g. built-in functions) and the locals `param1`, `param2`, and `x`. `y` should not be included, because a value is assigned to it not until line 4. Additionally to a display name, a LSP completion item can optionally contain a completion kind (e.g. variable, function, or class), a detail text for signature or type information, and a documentation. To provide these items, we suggest the following steps.

1. Parse the source code into a language-agnostic IR of the framework
2. Map line and column of the completion request to the IR
3. Derive lexical scopes from the mapped IR-element
4. Build completion items by gathering additional information for the obtained symbols of the scopes

In the first step of parsing source code into an IR of the framework, we can reuse the existing parser, which the language developer has to provide anyway. This parser will typically be syntax-error intolerant, so that we decided to support code completion only if the source code is parsable.

The second step requires, that selected elements of the IR contain references to their lexical sources, or at least a process to obtain these references afterwards. If we assume an AST as IR, relevant nodes could store start- and end-offsets of characters for the parsed source code. Then, the IR elements need to be searched for an element which maps best to the position of the completion request. Which one that should be, depends on the IR and the framework. A tagging mechanism could help to mark special IR-elements in a language-agnostic way, to allow filtering.

In the third step, we assume that the language implementation framework has a concept of scopes for the language implementations. For now, we are interested in lexical scopes only. Typically, there are some language built-ins which are always available, independent of the current context. They can be grouped in a global scope. The IR-element, matching the completion request position, can be used to derive local scopes together with symbols valid in these scopes. If we take an AST as IR example again, local scopes can be obtained by traversing the AST. All scope-building language elements, like functions, classes or blocks, can be collected along the way, starting at the node, which matches the completion request position, up to the root.

Finally, we create the completion items based on the symbols of the scopes. Depending on the meta information available for each symbol, we can add a completion kind, signature information for callables, or a documentation text.

Überleitung

3.2. Completion of Locals Using Run-time Information

A detailed code completion for globals, locals, and properties of language elements is often implemented in IDEs using type information. For programming languages with typed symbols, like Java, the required information can be easily derived from by parsing source code. The situation for languages with untyped symbols, like Python, is different, because the type depends on the associated run-time object.

There are language-specific approaches to infer types of variables for such languages , but these approaches ... .

ref

In this section, we describe an approach on how to enhance code completion for locals using run-time information. This approach will also build the basis for *completion of properties* described in Section 3.3.

far from perfect, difficult to built in a language-agnostic way, ...

Approach and Challenges The basic idea to get run-time values for locals at a specific position in the source code, is to execute that source code while observing the execution to collect run-time information at the desired position. This approach implies some preconditions. Language implementation frameworks need to provide a possibility to execute a program until a specific source code position is reached and then to inspect the current run-time state. This is already possible, if the framework provides a debugger for the developed languages. If present, we can reuse the same approach as used by the debugger to halt at a breakpoint and obtain locals and their corresponding run-time values from the active frame.

Challenges: Furthermore, executing arbitrary source code implies to think about how to deal with unexpected and unwanted side-effects during execution, the possibility of a long running or non-terminating execution, or the fact that the desired position might not be reached by the execution flow.

rewrite

Unreachable Source Sections Depending on the source code, there might be source sections which are unreachable by the normal execution flow of the current source code file. In our opinion, this is the normal case for file based programming languages. You typically split your program's source code into multiple files, having a limited set of files, which define execution entry points, which then import elements of the other files.

One of these entry points are unit tests. Programmers typically write them to ensure the desired behavior of their programs and avoid regression of bugs. Therefore, unit tests should cover most parts of the source code. For our approach of getting run-time information to enhance code completion is the systematic execution of unit tests a promising way. Most unit tests have a short execution time and are supposed to be free of unwanted side-effects – or at least have a clean-up routine included. We only have to provide programmers a way to tell our language server how to execute the tests. This could be done via a separate configuration file or via a special comment in the first or last lines of a source code file, which encodes a path to a runnable script. This idea is inspired by *type harvesting* [5], a practical approach to obtaining typing information in dynamic programming languages by exploiting unit tests.

Using the described approach, we have to care about unsaved changes. If we execute unit test, we have to ensure that they do not import files from the file system, if our server has newer, but yet unsaved versions. The LSP specifies no request sent from server to client to trigger saving files and we do not want to write the changes to the file system from server-side. Therefore, we suggest to redirect file accesses of files with unsaved changes to (virtual) files which contain the changes. This could be realized by installing a custom file system, if the framework supports that. Otherwise,

an enhanced code completion using run-time objects is only possible after saving all changes actively at client-side.

Because unit tests are not always available, especially if you start writing a small script or do not follow Test-Driven Development (TDD) principles , we fallback to executing the current source code, in which the code completion was triggered.

explain in background?

Sandboxing the Execution Environment Users would not expect that the source code they are currently working on will be executed when they trigger code completion. Therefore we suggest to create a sandboxed execution environment, when the execution of arbitrary source code is triggered implicitly. Therefore, the framework needs to provide an abstraction layer, which allows to control all I/O of the implemented languages. The most radical approach would be to forbid all I/O in our sandboxed environment. Unfortunately, our approach of using run-time information for code completion would thereby be limited to theoretical examples only. If the framework supports a fine grained control over I/O, we suggest to allow at least some operations. For standard I/O , we suggest to allow any kind of input, but redirect all output to "null". For file I/O, we suggest to simulate a read-only file system or, if supported, install a custom copy-on-write file system, which copies each file before modification to a sandbox and redirects further read and writes to that copies. For network I/O, we cannot make any assumptions concerning side-effects and suggest therefore to disallow all network I/O.

ist das das richtige Wort für console I/O

?

The situation is different, if users explicitly allow side-effects. This might be the case if they have defined an execution entry point, like a unit testing script, which performs only known and uncritical side-effects. We suggest therefore to provide a configuration option, to tell our language server, if sandboxing shall be deactivated when collection run-time information for code completion.

Long Running or Non-terminating Executions Users become impatient if the completion request takes to long to return a result. Unfortunately, the execution time of arbitrary source code might take an unpredictable amount of time or might not finish at all, for example due to an endless loop or blocking operations. Furthermore, the LSP does not support responding completion results incrementally. Therefore, the first thing we suggest, is to explicitly stop execution when the execution flow has reached the desired position in the source code and we have collected the needed run-time objects to build our completion items. Second, we suggest to add a general timeout to ensure an upper bound of execution time. In case of a timeout, no run-time information can be provided to enhance our code completion, but an endless execution can be avoided.

Caching Run-time Information Executing a script or unit tests for every *completion* request might yield no results due to long running executions and timeouts. Therefore we suggest to cache collected run-time information by storing a mapping from lexical source positions of IR-elements to collected run-time objects. Having such a data structure, we can use again the approach of finding an IR-element near the completion request's position to do a look-up in our mapping via the IR-element's

lexical source position.For simplicity and performance, we suggest to only cache the run-time information available during execution for every source section.

3.3. Completion of Properties

The second kind of code completion is the *completion of properties*.

- distinguish completion kinds -> completion trigger characters
- basically use same approach as for locals with run-time infos
- but new challenge: code not parsable
- solution: undo last change and parse again
- Find IR at specific source code position
- strategies: literal-approach, cached data in-line eval, run-to-section eval, fallback: global eval
- query found object for properties

listing 2 shows a Python code snippet, containing a class `Natural` with an instance variable `n` and a method `next`. In the last line, we assume that the dot was the last character inserted and a code completion request is triggered. We expect, that a completion result includes `n` and `next`.

Listing 2: A Python code snippet showing a property completion example.

```
1  class Natural:
2      n = 2
3
4      def next(self):
5          r = self.n
6          self.n = self.n + 1
7          return r
8
9  natural = Natural()
10 natural.
```

3.4. Transferring the Approach to Other Requests

- signatureHelp
- hover
- documentSymbol
- symbol
- definition
- references
- documentHighlight

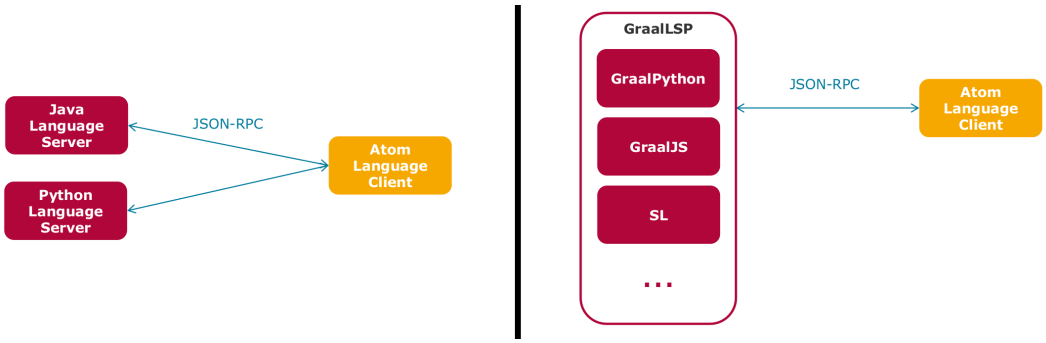


Figure 3.1.: lang specific server vs lang-agnostic server

old apporach In Chapter 1 we described the problem, that IDE-like tools are normally developed for a specific language only. Therefore, tooling support might be not the main focus of language developers, although they and their language users would benefit from them already during the language development process. As described, the Truffle framework has built-in support for language-agnostic tools and implementing a language server for Truffle could close an important gap of missing IDE-like tools.

The LSP enables arbitrary language clients to communicate with arbitrary language servers. Normally a language server provides support for one specific language only. Our approach aims to support multiple Truffle languages by build a language-agnostic language server using the Truffle instrumentation API. Figure 3.1 illustrates the difference.

We have to keep in mind, that LSP is an abstraction of typical IDE features for file-based programming languages. There are and will probably always be language-specific IDEs with a richer or more suitable feature set, because they can exploit language-specific knowledge. Therefore our approach will also be a trade-off. On the one hand, we can only access a limited language-agnostic API to build the language server. But on the other hand, the promising perspective to get IDE-like tools for Truffle languages with minimal additional implementation effort for language developers.

focus?

focus on high-level dynamic programming languages?

One of the greatest challenges for our approach is, that the ASTs of Truffle languages are run-time focused. Languages are implemented for a maximum of performance and information which are not relevant for execution are therefore dropped.

In the following sections we suggest, how the existing Truffle language and instrumentation APIs can be facilitated to provide the needed data for LSP requests and notifications. If needed, we also suggest small API extensions to get access to additional language-specific information, which have not been exposed yet. This is mainly due to fact that the existing tools are, like the ASTs, mostly run-time focused and do not require these information.

Table 3.1 gives an overview over the most interesting requests and notifications of the LSP version 3.7.0 which we cover in our approach.

Table 3.1.: Selected LSP features and their descriptions

Request/Notification	Description	Section
initialize	First request containing client capabilities. Response contains server capabilities.	3.5
didOpen	Notification sent from client to sever that a file was opened. Contains URI and content.	3.6
didChange	Notification sent from client to sever that a file has changed. Contains URI and changes.	3.6
diagnostics	Notification sent from server to client to display errors, warnings or hints.	3.7
completion	Completion request at caret position. Can be triggered automatically via registered completion trigger characters.	3.9
signatureHelp	Request to find signature information (e.g. parameter names and details for a function) at caret position. Is triggered via registered signatureHelp trigger characters.	3.12
hover	Request for information at caret-hovering position.	3.13
documentSymbol	Request for symbols of a specific source file (function names, variable names, module names, etc.).	3.14
symbol	Like documentSymbol, but for all files in the client workspace.	3.15
definition	Goto-definition request for functions, variables, etc.	3.17
references	Request to find all references of a symbol.	3.18
documentHighlight	Request to highlight all ranges related to a symbol (e.g. variable reads and writes).	3.16
codeLense	Request to provide executable actions at arbitrary source locations.	3.19

3.5. Language Server Initialization

The first request sent from the client to the server is the *initialize* request. Among other parameters, it contains client capabilities, telling the server which LSP features the client supports. The server responds to this by sending its own capabilities. If a server supports the *completion* and *signatureHelp* requests, the response has to include trigger characters[1] which tell the client when it shall send a *completion* or *signatureHelp* request automatically. A language-specific language server implementation can provide these characters statically. In our language-agnostic approach, where Truffle languages can be installed dynamically, we have to collect these characters for all available languages during server initialization. Using the Truffle instrumentation environment, we can get a list of all registered languages, but there is no API to obtain trigger characters for *completion* and *signatureHelp* requests for a language. Therefore, this is our first Truffle API extension, which we suggest.

After receiving an *initialized* notification from the client, both sides are ready for further requests and notifications.

[1] A trigger character is actually a string and can consist of multiple character symbols, but some clients support only a single character symbol yet.

3.6. Parsing Source Code

To parse source code in Truffle, we need to define a `Source` object, containing the language and a reference to the actual source code, typically via a file. The language can be derived from the supported media types of the language. The LSP requests *didOpen* and *didChange* contain an URI pointing to the concerning source file, which we can use to find the required media type. The *didOpen* request also contains the file content. The *didChange* request contains only incremental changes if configured during initialization. Therefore we have to continuously store the content and apply all incoming changes to keep track of the client-side file state.

paragraph feels like to much implementation?!

The *initialize* request can contain a workspace URI. If present, we parse all files in the workspace folder and its subfolders during initialization, which match one of the media types of the installed Truffle languages.

If parsing succeeds, Truffle's parsing result is a `CallTarget`, ready to be executed. It holds a reference to a root node of a run-time-focused AST.

In case of a parsing error, we can notify the client using the *diagnostics* notification.

3.7. Diagnostics

Truffle languages typically throw a `TruffleException` in case of a parsing error. The parsers are not designed to be syntax error tolerant and will stop parsing at the first syntax error. A `TruffleException` can have a reference to the location where the exception occurred in the AST. If available, we map these information to line and column of the original source code and send a *diagnostics* notification to the client. If the location is not available, we use the first column in the first line as reference for the notification. The error message needed for the notification is also derived from the `TruffleException`.

3.8. Mapping Caret Position to AST Node

Most LSP requests are context-sensitive and include therefore line and column as reference. The language server has the responsibility to map these information to data structures of the language. For Truffle, this means mapping line and column to AST nodes via their associated source section. The challenge for our language-agnostic language server is, that the Truffle framework leaves the creation of the run-time focused ASTs completely to the language implementation. We cannot simply traverse an AST by starting at the `CallTarget`'s executable root node which we get as parsing result. There might be more root nodes, which will be executed dynamically at run-time, depending on the language implementation. However, we can use the Truffle instrumentation environment to access all already loaded nodes which have a source section, under the condition, that the source code is parsable.

Depending on the use cases described in the following sections, we will be either interested in a node directly at the caret position (*hover*, *definition* and other requests)

or a node near the caret position (*completion* request for globals and locals). To find these nodes in a language-agnostic way, we have to define two assumptions.

Eager Source Code Parsing First of all, we assume, that parsing a Truffle `Source` object is done when we call the corresponding API and not lazily when the `CallTarget` is executed. Otherwise no nodes and source sections will be available via the Truffle instrumentation environment.

Consistent Source Section Hierarchy We assume furthermore, that source sections of ASTs follow the rule: if a node has a source section and is a child of another node, then the source section of the child is fully contained in all source sections of its direct parent or indirect parents. A parent without a source section in this hierarchy is skipped. Visual Studio Code for example builds code folding and the symbol outline based on the containment of ranges in other ranges.

3.9. Static Completion of Globals and Locals

We believe that a context-sensitive code completion is one of the most important features to increase productivity of programmers. It suggests valid symbols regarding the current scope and thereby supports programmers to explore, for example, the built-in functions of a language, read their documentation or select predefined code snippets.

For Truffle, we have to distinguish between two kinds of code completion, both depending on the current caret position context. The first one we call *completion of globals and locals*, described in this section. The second one we call *completion of object properties*, described in Section 3.11.

Completion of globals and locals is the completion you typically get in IDEs when the caret is placed in an empty line and the user presses a keyboard shortcut, e.g. `Ctrl+Space`. With *globals* we refer to all the symbols which we can derive from global scopes of languages and with *locals* we analogously refer to all symbols which we can derive from local scopes of languages. In the Python programming language for example, globals are built-in functions or built-in classes. Locals of a function scope are function parameters or local variables. Locals of a class scope are fields or methods. Figure 3.2 exemplarily shows a language-specific code completion for globals and locals of a Python file in Visual Studio Code.

If the assumptions in Section 3.8 (eager source code parsing and a consistent source section hierarchy) are true, we can provide *static* code completion for globals and locals by using existing Truffle APIs. With *static*, we refer to a code completion, where we can only access lexical scopes to find locals. An approach to use run-time information for code completion, e.g. via a frame, is explained in Section 3.10.

Statically Deriving Local Scopes From a Node Global scopes are always accessible via the Truffle instrumentation environment. To find the relevant local scopes statically, we need to find a node which has a source section *relevantly near* the client's

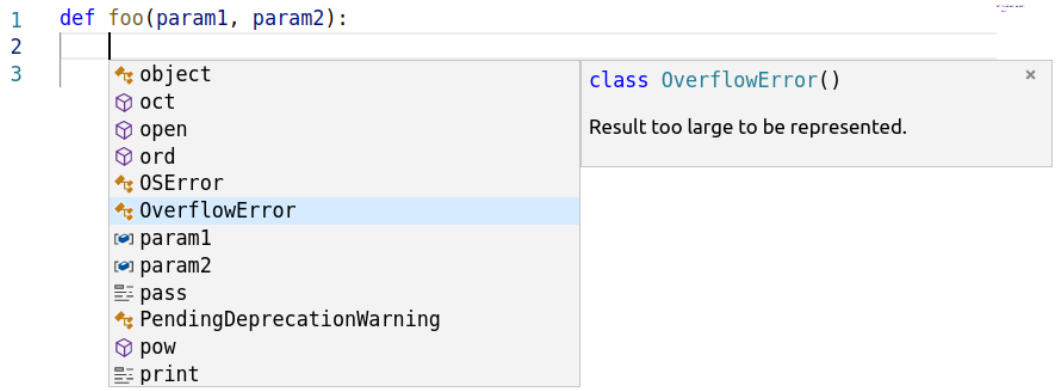


Figure 3.2.: An existing Visual Studio Code extension for the Python language showing the completions of globals and locals available in a function scope, together with reserved keywords.

caret position. In the following we use a Python code snippet to explain what we mean by *relevantly near*.

Listing 3: An example Python code snippet showing two functions to explain scopes.

```
1  def foo(param1, param2):
2      x = 42
3
4      y = param1 + param2
5
6      print(x * y)
7
8
9  def bar():
10     return 42
```

label counter wrong?

listing 3 shows Python source code with a function definition `foo`, containing two parameters and two local variables in its scope and a function `bar` without definition of locals. Imagine that the caret is placed in the empty line 3 between the two variable assignments and code completion is triggered. The caret is clearly placed inside the scope of the function `foo` and we would assume that the code completion suggests, among globals, the two parameters `param1` and `param2`, and the local variable `x`. The local variable `y` is also in the function scope of `foo`, but is not written until line 4 and should therefore not be included in the suggestions. Whereas, if the caret is placed in line 5, `y` should be included.

As already mentioned, Truffle cannot derive scopes directly from the caret position, but from a node. The difficulty is, that the scopes, which the Truffle API derives from a node, contain variables valid *at* that node. This means translated to our Python example, that if we use the node representing `x = 42` in line 2 to derive local scopes from, the scopes do not include the variable `x`. One could say, that we get all locals valid before the node's source section starts. Therefore we have to decide in

a language-agnostic way, which node is near[2] the caret and provides the relevant information we need for the code completion.

Imagine code completion is triggered in line 3. Then the most inner node, which contains the caret, could be the function definition of `foo` ranging from line 1 to 6. The nearest node behind the caret could be the assignment of `y` in line 4 and the nearest node before the caret could be the assignment of `x` in line 2.

We suggest finding the node *relevantly near* the client's caret position following the rule:

1. If the caret is not contained in a node, we are not in a local scope. No completion of locals necessary. (Example: caret at line 8)
2. If contained, pick the node which is nearest behind the caret position and which is also an (indirect) child of the node which contains the caret. (Example: caret at line 3, picking node `y = param1 + param2`)
3. If not found, pick the node which is nearest before the caret. (Example: caret indented at line 7, picking node `y = param1 + param2`)
4. If still not found, use the node which contains the caret.

At this point, we have to note that this rule is only a best-effort approach. Which locals are included in the lexical scopes provided via the Truffle API is depending on the language implementation. None the less, we are able to provide the needed information for a language-agnostic, static code completion of globals and locals with this approach.

Querying Properties of Scope Values For globals, we can not only access their names via the Truffle API, but retrieve additional information by querying the actual objects which are associated with these names in the global scopes. Based on the result, we can set the completion item kind supported by LSP, such as *variable*, *function*, or *class*.

A property, which is not supported by the current Truffle API, is documentation. Therefore, we propose to add a new Truffle API. For Python, this API could return the documentation string for functions, classes or modules.

To sum it up, with the proposed extension for documentation, we can support static code completion for locals without further details, but for globals with completion kind and documentation.

3.10. Completion of Locals Using Run-time Information

A detailed code completion for globals, locals, and properties of language elements is often implemented in IDEs using type information. For statically typed programming languages, like Java, type information can be easily derived from by parsing source code. The situation for dynamically typed languages, like Python, is different. The type of a Python variable is depending on its assigned value at run-time. There are

[2] For simplicity, we say in the following *a node near/after/before/containing the caret* instead of *a node which has a source section near/starting after/ending before/containing the caret position*

ref

language-specific approaches to infer types for dynamically typed languages , but these approaches do not match the language-agnostic requirements of the Truffle instrumentation API.

In this section, we describe an approach on how to get and use run-time information to enhance code completion for locals. This approach will also build the basis for *completion of object properties* described in Section 3.11 and further requests described in later sections.

Approach and Challenges The basic idea to get access to run-time information (e.g. a frame) at caret position is, that we trigger source code execution while using Truffle instrumentation to listen for execution events occurring at a source section near the caret position. This approach implies some preconditions and challenges. We have to think about unexpected and unwanted side-effects during execution, the possibility of a long running or non-terminating execution, and the fact that the caret position might not be reached by the execution flow.

is embedding-API the right term?

Sandboxing the Execution Environment The Truffle framework has an embedding-API which allows developers to configure sandboxing for the Truffle execution environment. At the moment, it supports restriction of the file system only, but in future versions of Truffle, the restriction of network I/O shall also be supported. It is also possible to install our own file system proxy, which enables having fine grained control over what Truffle languages are allowed to access and what not. We could therefore install a copy-on-write file system, which copies each file which shall be changed to a sandbox and redirects further read and writes to that copy. For now, we simply use a read-only file system, which prevents unwanted file changes outside the Truffle execution environment, when executing source code in our language server.

Long Running or Non-terminating Executions A user, who triggers code completion, becomes impatient if the completion request takes to long to return a result. The execution time of arbitrary source code might take an unpredictable amount of time or might not finish at all, for example due to an endless loop or blocking operations. Therefore we have to explicitly stop execution when the execution flow reaches the desired source section. As a fallback, a time-out should ensure an upper bound of execution time by interrupting the execution. In that case, no run-time information can be provided for enhanced code completion, but an endless execution can be avoided.

Unsaved Changes The language client might have unsaved changes in opened files. These changes need to be considered, when we want to collect run-time information by executing a script, which imports other sources. These imports need to access the source code state as it is seen by the user in the language client and not the state in the file system. The LSP provides no request sent from server to client to trigger saving files and we do not want to write the changes to the file system from server-side.

Therefore we decided to adjust our read-only file system, so that it redirects the file accesses to a file with unsaved changes to a (virtual) file including the changes.

Unreachable Source Sections Depending on the source code, there might be source sections which are unreachable by the normal execution flow of the current source code file. Imagine, the Python code defined in the already mentioned listing 3, is the source code of a python file. If we execute that file, the two function definitions of `foo` and `bar` will be executed, but their function body will not be executed. Their is no function call to `foo` or `bar` defined in that file. Therefore, listening for execution events of the Truffle instrumentation environment to enhance our code completion in lines 2 to 6, which is the function body of `foo`, will not work. In our opinion, this is the normal case for file based programming languages. You typically split your program's source code into multiple files, having a limited set of files, which define runnable entry points, which then import elements of the other files.

One of these entry points are unit tests. Programmers typically write them to ensure the desired behavior of their programs and avoid regression of bugs. Therefore, unit tests should cover most parts of the source code. For our approach of getting run-time information to enhance code completion is the instrumented execution of unit tests a promising way. Most unit tests have a short execution time and are supposed to be free of unwanted side-effects – or at least have a clean-up routine included. We only have to provide programmers a way to tell our language server how to execute the tests. This could be done via a separate configuration file or via a special comment in the first or last lines of a source code file, which points to a runnable script. This idea is inspired by *type harvesting* [5], a practical approach to obtaining typing information in dynamic programming languages by exploiting unit tests.

Because unit tests are not always available, especially if you start writing a small script or do not follow TDD principles , we use the current file, in which the code completion was triggered, as fallback for a runnable entry point.

explain in background?

Caching Run-time Information Executing a script or unit tests for every *completion* request might still be to expensive and hinder the writing flow of the programmer. Therefore we need to cache collected run-time information. We suggest to use a simple mapping from source sections to frame snapshots which we set up using Truffle instrumentation. We know that different execution branches might reach the same source section with a frame, which has not only stored different values of the same type, but also values of different types. But for simplicity and performance, we decided to only cache the first frame available during execution for every source section.

Having such a data structure, we can use the approach described in Section 3.8 to find a node near the caret an check via its source section for collected frames. If there is one, we can derive local run-time scopes from it. If there is none, we have to assume that the source section was not covered and use local lexical scopes as fallback.

3.11. Completion of Object Properties

In the previous sections we described the completion of globals and locals using static and run-time information. This section explains an approach to *complete object properties* in a language-agnostic way. With *completion of object properties* we refer to the typical completion you get in IDEs when inserting a trigger character, like a dot in Python or Java, after a variable. This way, you get suggestions for named properties available on the object bound to that variable. What a property actually is, depends on the Truffle language implementation. For a Python or Java object, properties could be all fields and methods which are defined by the object's class and super-classes. For a non-object-oriented language, this kind of completion might not be supported or be limited to special language elements, which understand some kind of *property* semantic. listing 4 shows a Python code snippet consisting of a class definition `Natural` with a field `n` and a method `next`. In the last line, a code completion could be triggered by an IDE automatically after the dot character was inserted. We would assume, that such a completion suggests the field `n` and the method `next` at least.

Approach and Challenges Truffle provides an API to list all property names of a language object and read or write the actual values associated with these property names. To use this API, we have to solve some challenges. First of all we have to detect that an object property completion shall be triggered instead of a completion of globals and locals. Second, we have to identify the language element, i.e. a Truffle AST node, for which the completion of properties is desired. Finally, we have to obtain the actual Truffle object associated with that node, which might be available only at run-time.

Detecting an Object Property Completion To distinguish a completion request for globals and locals from one for object properties, we can use the API extension already suggested in Section 3.5, which allows us to ask a Truffle language for its completion trigger characters. Because the *initialize* response contains the trigger characters of all installed languages, the client will automatically trigger completion requests for all these characters. Therefore we have to check every request, if the trigger character belongs to the current language. If not, we have to skip the request.

The completion trigger character is only included in the request if the completion was triggered automatically and the client supports that optional feature. Therefore we have to check manually. To do so, we assume, that when a completion request arrives, we can distinguish the two kinds of completion by comparing one or multiple characters directly in front of the caret with the registered trigger characters of the current language. If there is a match, we try an object property completion.

Finding the Desired Node To find the AST node for which the completion of object properties is desired, the source code needs to be parsable. We assume, that this is not always the case when a completion is triggered, because an inserted completion character might cause a syntax error. Therefore we suggest to revert the last textual

change which was sent from the client via the *didChange* notification and then try to parse the reverted source code.

Listing 4: A Python code snippet showing an object property completion example.

```
1  class Natural:
2      n = 2
3
4      def next(self):
5          r = self.n
6          self.n = self.n + 1
7          return r
8
9  natural = Natural()
10 natural.
```

As an example, we refer again to listing 4. The source code shown is no valid Python source code because of the last dot character. Removing this dot makes the code parsable again.

Now that we have a first approach to get parsable source code again, we can use the approach described in Section 3.8 to find a node directly in front of the current caret position.

Obtaining the Actual Properties The next step we have to do is getting the actual object associated with that node, which might be available only at run-time, to finally query it for its properties. The approach is basically the same as for the completion of locals using run-time information, described in Section 3.10. But there are some optimizations we can do.

First of all we can optimize property completion for literals. We can test the node in front of the caret, if it represents a literal and then query it for the actual literal object. This literal object can be used to obtain properties, if the language supports it. In some languages, for example Python, Java or Javascript, object property completion is possible for string literals.

If the literal-approach returns no results, we can check if there are already cached run-time information available (see Section 3.10). These information can be used by the Truffle instrumentation environment to perform an in-line evaluation of the source code associated with the node in front of the caret. As a prerequisite, a Truffle language needs to support in-line parsing of source code. The evaluation result can then be queried for properties.

If using cached run-time information returns no results, we use the actual approach described in Section 3.10, i.e. we execute the current script or unit tests and listen for execution events concerning the source section of the node in front of our caret. Translated to our Python example in listing 4, where we assume that a completion request was triggered by inserting a dot character in the last line, we are interested in the return value of the expression `natural` in front of that dot. Therefore we listen for execution events which are associated with the source section of `natural`. The resulting object can be queried for properties.

If there are still no results, because the source section might not be reachable for example, we can use a last fallback. We try to evaluate the source snippet associated

list problems? context-depedent code might be evaluated in global scope yielding wrong results etc.

with the node in front of the caret in the global scope. This will at least provide some property completion for globals and context-independent expressions.

To sum it up, with the approach described in this section, we are able to provide a basic, language-agnostic code completion for object properties. This approach includes reverting source code to a parsable state, executing that source code in a sandbox to listen for specific execution events using Truffle instrumentation, and querying the obtained run-time language object for its properties.

3.12. Signature Help

The *signatureHelp* request is sent from client to server to obtain signature information (e.g. function parameter names) at the caret position. The trigger characters for this request need to be defined in the response of the *initialize* request. Therefore, we have related challenges like the *completion* request for object properties. First of all, we have to distinguish if the trigger character, which caused the request, is belonging to the current language. If not, we can skip the request. Second, the sole insertion of the trigger character might cause a syntax error. Third, we have to obtain the actual run-time object to query it for signature information. These challenges can be analogously approached like those for the completion of object properties in Section 3.10, i.e. bringing the source code into a parsable state, finding the node at the caret position, and executing the source code until we have found the desired run-time object.

Unfortunately, there is no Truffle API yet, which provides signature information for an object. Therefore, we suggest to add a new Truffle API. Furthermore, we assume, that only callable elements of a language can have a signature. To mark these elements of a language, Truffle provides a `CallTag` for AST nodes. We can use this tag as a filter criterion for our search for the node at the caret position.

3.13. Hover Information

The *hover* request is sent from client to server to obtain information at the current (mouse) caret position. This request has the same prerequisite as the other context-depending requests. The source code needs to be parsable to find a node at the caret position. Having a node, we can check if the node's source section is associated with run-time information (see Section 3.10). If so, we can use these information (e.g. a frame) to build the *hover* response. If the hovered node represents a variable access, we can use the frame to obtain the assigned value and include the string representations of that value and its meta-object in the response. Otherwise, we try to perform an in-line evaluation of the source code associated with the node. If the result is an object which supports the suggested API to obtain signature information (see Section 3.12) or to obtain documentation (see Section 3.9), we can include the signature or documentation, for example for a Python function, in the *hover* response.

For language developers or interested language users, we suggest to provide static hover information concerning the AST node at the caret position if no run-time

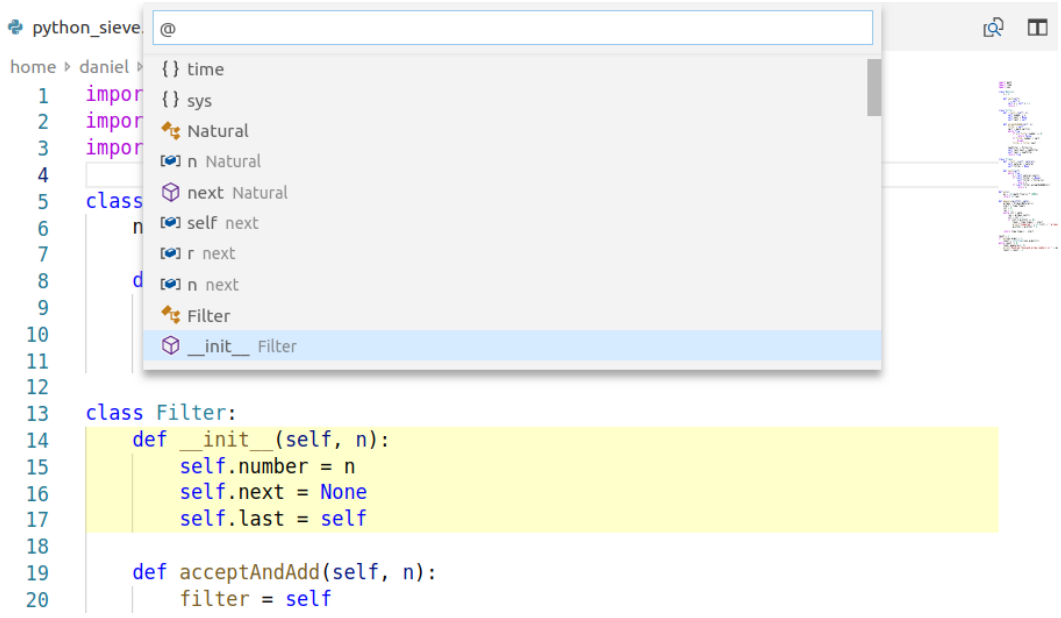


Figure 3.3.: A Visual Studio Code extension for the Python language showing all detected symbols in the current document. There are different icons for modules, classes, functions and variables.

information are available. This feature, which can be activated via an option, can be used to inspect the AST structure of a language and learn about which language elements in the source code are mapped to which kind of AST nodes.

3.14. Document Symbols

The *documentSymbol* request is sent from client to server to get all symbols available in a specific source file. Each symbol in the response must be associated with a source range. If the symbol, for example, relates to a function definition, the range of the definition including the function body could be taken as the source range. Additionally, a symbol kind can be added for each symbol in the response. Figure 3.3 shows how the *documentSymbol* response is displayed in Visual Studio Code with an already existing language-specific Python extension.

To support the *documentSymbol* request for Truffle languages, we have to think about which kind of symbols we can obtain with the current Truffle API. With the instrumentation environment we can access all loaded nodes which have a source section. We can filter them based on their meta information to find all root nodes. Because a root node is executable, we assume that the name of the root node is a valid symbol of a callable element of the corresponding Truffle language. However, to add a symbol kind we need more meta-information about the nodes.

Tagging Nodes with Meta-information To get the required meta-information, we suggest to add a new `Tag` for nodes introducing new symbols in a scope. There are already some tags existing to mark statements, expressions, calls, and root nodes.

They are mainly used to implement debugger features like stepping. Using again the example of the Python language, the tag we suggest could be used to mark function and class definition nodes, first occurrences of a variable assignment node (Python has no variable declarations), or import nodes. This way we could support the *documentSymbol* request in a language-agnostic way with results equal to those language-specific ones already shown in Figure 3.3.

3.15. Workspace Symbols

The *symbol* request is the union of *documentSymbol* requests for all files in the workspace. Therefore, if we apply the approach explained in Section 3.14 for *documentSymbol*, we can easily add support for the workspace-wide *symbol* request.

3.16. Document Highlight

The *documentHighlight* request is sent from client to server to get a list of ranges, which shall be highlighted in the client. Additionally, we can define if the highlighted range is a read or write of a symbol. We want to support highlighting of variable reads and writes for now. Unfortunately, there is no Truffle API yet to filter for nodes representing a read- or write-access of a variable. Therefore, we suggest to add two new tags to mark nodes which read or write a variable. These nodes also have to expose the identifier of the variable which they refer to. Having this, we can check if the node at the current caret position is tagged with one of these new tags. Then we can start a static search in the local scopes derived from that node for other tagged nodes with the same variable identifier. For every matching node, we add and response item, which includes the node's source range and the highlighting kind.

3.17. Goto-definition

The *definition* request is sent from client to server to find the definition location of the symbol at the caret position. This feature is known as goto-definition or jump-to-definition in most IDEs. We want to support goto-definition for callables and variables. Like for many other requests, the code needs to be parsable to find the nearest tagged node at the caret position. Either tagged with the existing `CallTag` of Truffle, or with the tags suggested in Section 3.16 to mark read- or write-accesses of variables.

For callables we propose to use the same run-time approach as for the run-time information based completion described in Section 3.10. This way, we get access to the callable run-time object. The Truffle API allows us to obtain the corresponding root node of that object and thereby the source section. We use this section to create the response for the definition request.

If the run-time approach yields not results, we can use a static fallback search, based on meta-information of nodes. We already described this approach in Section 3.15

to find all symbols in the workspace. We can filter these symbols for those matching the identifier of our callable node. For non-unique identifiers, this might yield many source sections, which are not the actual definition location. But it is better to provide the user a list of potential matches instead of providing nothing. Of course, regardless if an identifier is unique or not, the actual definition location might not be included in the suggestions at all. This is again a static, language-agnostic best-effort approach.

For variables, we assume that the first write of a local variable is its definition location. This way, we can reuse the approach described in Section 3.16, to find all other occurrences of a local variable and pick the first write as definition location.

3.18. Finding All References

The *references* request is sent from client to server to get all locations in the workspace, where the symbol at the caret position is also referenced. For now, we support a reference search for local variables and callable objects. As for many requests described before, the code needs to be parsable so that we can find a node a the current caret position.

For callable objects, we can search for nodes tagged with Truffle's `CallTag`. If there is such a tagged node at the caret, we can obtain its symbol and search all already loaded sources for other tagged nodes with the same symbol. Then we return all source section ranges of the matching nodes in the response.

For local variables, we need the tags suggested in Section 3.16 to search for nodes which represent a read- or write-access to a variable. If there is a node tagged with one of these two tags at the current caret position, we can search for all other variable references as described in Section 3.16 for the highlighting of variable reads and writes.

3.19. Triggering Run-time Information Collection

In Section 3.10 and the following ones, we described how to use cached run-time information for code completion, hover details and other requests. The question is, when to initialize and renew this cache. Therefore, we use the *codeLense* request, sent from client to server, to provide an interface for LSP-client users, so that they can trigger the renewal of the cache when they considered it reasonable. This could be the case, when the hover request returns incomplete or no results anymore.

The *codeLense* response includes a list of commands associated with display names and source ranges. It allows interspersing contextual actions directly into the source code. In a Visual Studio Code extension, it is used for example to insert a clickable link above a TypeScript function, if it violates some predefined code metrics. Clicking the link shows the metrics. We use it to insert such a code lense in the first line of the code, which will trigger the execution of the current source code or another script to collect run-time information.

Migrating Cached Information One challenge, we have not discussed so far, is that the cached run-time information are bound to specific source sections of AST nodes. Every time a *didChange* notification arrives, one or more sections of the source code will change. Depending on the change, the cached run-time information might become invalid for some source sections, but others might still be useful.

Imagine, we have collected run-time information for the Python code snippet in listing 5. Now, we edit this source snippet by inserting some code in line 3, which triggers an object property completion request for variable `anObject`. The collected run-time information in lines 1 and 2 are still valid, so that we can use them to perform an in-line evaluation for `anObject` to obtain its properties (see Section 3.11).

Listing 5: A Python code snippet example to illustrate migrating cached run-time information.

```
1 x = someCalculation(42)
2 anObject = expensiveOperation(x)
3
4 doSomething(y)
```

Imagine, the next change inserts a new empty line between lines 1 and 2. Our mappings of source sections to run-time information for all lines greater 1 are all invalid now. Therefore we suggest a simple approach to migrate these information. In the current case where a new line was inserted, this would be to simply adjust the off-by-one mismatch in the line numbers of the source sections following after that line. This is possible, if we address a source section only by its start and end line numbers and column offsets. We assume, that most edits will affect only their current local scope and there only the current line, so that we can apply the following rules for our mappings.

1. If a source section's end line is smaller than the start line of the change, we do nothing.
2. If a source section's start line is greater than the end line of the change, we check if the change adds or removes lines. If so, we adjust start and end line of the mapping by that difference.
3. If the range of the change is fully included in a source section, we adjust only the end line of the mapping by the difference of added or removed lines.
4. In all other cases where start or end lines of mapped source section and change overlap, we assume that the mapping is not valid anymore.

This way, we can migrate cached run-time information mappings for simple changes. But we can only assume that the adjusted mappings are still valid. The user has to trigger a new collection run manually. For example, if changes alter the execution flow, names of variables or their types.

4. Implementation

In this chapter, we describe how we have implemented our prototype Graal-LSP based on the approach described in Chapter 3.

4.1. Architectural challenges

- Building tool for Truffle = using instrumentation API
- LSPInstrument loaded dynamically via classpath
- onCreate() we get an Env -> our entry point to parse code, find scopes etc.
- For getting run-time infos we execute code -> sandboxing needed -> via creation of Truffle Context
- Truffle instrument cannot create new Contexts
- Need to use embedder-API -> write own launcher, create engine ourselves
- set read-only file-system on context builder
- Provide a callback to our instrument to create new sandboxed Contexts
- lsp4j -> own threads to handle requests
- Threads need to be activated for Contexts - multi-threading is not always supported
- Our decision -> we have only one worker thread which enters a Context once
- lsp4j thread puts work in a queue which is polled by the context aware worker thread
- lsp4j threads await result via Futures

4.2. Language Server Initialization

- new API for trigger chars to feed init request -> env.getCompletionTriggerCharacters()

4.3. Parsing Source Code

- env.parse(Source)
- Source is built from source code and language
- Some lsp clients send a languageId, some a media type
- 1. env.getLanguages().containsKey(langId)
- 2. Source.findLanguage(String mimeType)
- 3. Source.findLanguage(URL)
- -> show source code?
- Track state -> didChange sends incremental changes
- didChange -> need to also update coverage data

4.4. Diagnostics

- TruffleException -> ast node -> source section -> map to lsp4j Range (zero- vs one-based lines)
- LanguageClient.publishDiagnostics(PublishDiagnosticsParams)

4.5. Mapping Caret Position to AST Node

- zero- vs one-based lines
- Truffle Source counts new-line char at end not as additional line -> often code completion triggered in new, empty line
- env.getInstrumenter().attachLoadSourceSectionListener(filter, listener, true)
- check node.getSourceSection() line and offset to find nearest

4.6. Static Completion of Globals and Locals

- code snippet: findNearestNode() ?
- env.findTopScopes(langId)
- env.findLocalScopes(node, frame=null)
- Scope.getVariables() returns a TruffleObject
- ForeignAccess.sendKeys(Node keysNode, TruffleObject receiver, boolean includeInternal)
- for globals:
  - ForeignAccess.sendRead(Node readNode, TruffleObject receiver, Object identifier)
  - completion item kind: ForeignAccess.sendIsExecutable(Node isExecutableNode, TruffleObject receiver) and ForeignAccess.sendIsInstantiable(Node isInstantiableNode, TruffleObject receiver), fallback CompletionItemKind.Variable
  - new API: env.boxPrimitive(Object) and GetSignature message for completion details
  - new API: GetDocumentation message for completion item documentation

4.7. Completion of Locals Using Run-time Information

- sandboxing: Context.Builder.fileSystem(FileSystem fileSystem)
- unsaved changes redirection in file system by callback to instrument
- long running ops: throw EvaluationResultException (which is a ThreadDeath exception)
- long running ops: time-out not yet implemented
- unreachable source sections: in-line comment to a run script
- using cached infos: Env.parseInline(Source source, Node node, MaterializedFrame frame) + insert into coverage data node + node.execute(frame)
- details created like in static completion of globals

4.8. Completion of Object Properties

- detecting: env.getCompletionTriggerCharacters(langId) comparing last change
- handling syntax errors -> we always store the last edit and then replace the last edit range with an empty string
- find nearestNode with CONTAINS_END
- strategies
  - literal -> node.getNodeObject() and check for "literal" key
  - using cached infos: Env.parseInline(Source source, Node node, MaterializedFrame frame)
  - run-to-section:
    - create SourceSectionFilter for node's source section bounds (cannot use section directly - might be source loaded via TruffleInstrument)
    - env.getInstrumenter().attachExecutionEventFactory(SourceSectionFilter eventFilter, SourceSectionFilter inputFilter, ExecutionEventNodeFactory factory)
    - ExecutionEventNode.onReturnValue(VirtualFrame frame, Object result)
    - throw new EvaluationResultException(result);
    - Errors during execution -> if TruffleException make Diagnostics warning
  - globals: eval node.getSourceSection().getCharacters()
- details created like in static completion of globals

4.9. Signature Help

- run-to-section but with inputFilter
- we assume code is parsable because of smart character insertion "()"
- SourceSectionFilter set to CallTag annotated node
- To get the callable object, we listen for intermediate results of expression calc using ExecutionEventNode.onInputValue(VirtualFrame frame, EventContext inputContext, int inputIndex, Object inputValue)
- ForeignAccess.send(GET_SIGNATURE, (TruffleObject) result);
- ForeignAccess.sendInvoke(INVOKE, (TruffleObject) signature, "format")
- signature help param index: getNodeObject() look for "numberOfArguments"

4.10. Hover Information

- cached frame 1.: frame.getFrameDescriptor().getSlots().stream().filter(slot -> slot.getIdentifier().equals(textAtH -> null);
- cached frame 2.: Env.parseInline(Source source, Node node, MaterializedFrame frame) + insert into coverage data node + node.execute(frame) <- same as for completion of locals using cached infos
- fallback (if option enabled): show AST details (class name and Tags)

4.11. Document Symbols

- SourceSectionFilter DeclarationTag + uri of current file
- getNodeObject() -> look-up keys: name, kind, container
- mapping to lsp4j SymbolKind via lower case name matching
- fallback: SourceSectionFilter StandardTags.RootTag + RootNode.getName() + default symbol kind: Function

4.12. Workspace Symbols

- same as documentSymbol, but no uri restriction in SourceSectionFilter

4.13. Document Highlight

- find node at caret: StandardTags.WriteVariableTag.class, StandardTags.ReadVariableTag.class
- getNodeObject() -> key look-up: name
- env.findLocalScopes(node, null) or env.findLocalScopes(node, frame) if cached frame available
- iterate scopes + scope.getNode().accept(NodeVisitor) -> search for all ReadVariableTag/WriteVariableTag annotated nodes

4.14. Goto-definition

- find node at caret: SourceSectionFilter with CallTag or ReadVariableTag (no WriteVariableTag, because source sections of write include assignments in Python -> leads to unexpected behavior <- should we change this?)
- CallTag: run to section with inputFilter like signatureHelp -> Env.findSourceLocation(LanguageInfo language, Object value)
- ReadVariableTag: like documentHighlight via scopes and visitor, but interpret the first write node of the most outer scope as declaration

4.15. Finding All References

- find node at caret: SourceSectionFilter with CallTag or ReadVariableTag or WriteVariableTag
- CallTag: env.attachLoadSourceSectionListener with SourceSectionFilter for CallTag, then match names
- ReadVariableTag/WriteVariableTag: like documentHighlight

4.16. Triggering Run-Time Information Collection

- command registration
- CoverageEventNode

- SourceSectionFilter for language
- onEnter() -> flag entered = true
- copy frame and descriptor to snapshot state
- store these together with CoverageEventNode

4.16.1. LSP client - VSCode extension example

4.17. Requirements on Truffle Language Implementations

- Implement existing API:
  - ExpressionTag + StatementTag
  - findTopScopes()
  - findLocalScopes()
  - findMetaObject()
  - findSourceLocation()
  - parseInline()
- Implement proposed/new API:
  - getCompletionTriggerCharacters()
  - getSignatureHelpCharacters() (TODO)
  - boxPrimitive()
  - GetSignature custom message (+ format invoke + getNodeObject() "numberOfArguments" look-up)
  - GetDocumentation custom message
  - DefinitionTag + ReadVariableTag + WriteVariableTag

5. Experience Report

Eat your own dog food Hover: We used the static hover information ourselves during implementation.

GraalPython, SL, (GraalJS), FastR usage check

- matrix of Graal-LSPfeatures x Truffle languages
- list SLOC for adjustments of these langs
- matrix of Graal-LSPfeatures x Truffle languages without language adjustments? Should be only completion of globals/locals and static stuff possible -> or add a footnote in the first matrix, to mark that it is only working with adjustments
- include screenshots of VSCode for important LSP features, put rest in appendix?

Completion response time

Between IDE and Language Workbench Language Workbench focus/aim: Efficient language (DSL) development, performance is not main focus vs. Graal/Truffle aim: Efficient execution

Assumption: Language Workbenches use static information only, our Truffle-based solution uses dynamic type information

5.1. Limitations of Graal-LSP

- Server: Only TCP (connecting to already running server), no std-in/-out impl
- Truffle: Only first syntax error is reported. Lang parsers fail on first error
- Cancellation of requests not yet supported
- also, this in-line eval is interpretation only
- sandboxing limited to fie I/O + read-only FS atm
- limitations due to caching of last nested context - show example of dirty context, e.g. function renamed
- Using cached run-time infos: we have frame but to do in-line eval we call node.execute(frame) ourselves without having a valid execution stack there -> if the execution relies on the stack, it will not work
- type harvesting: we take a snapshot of the frame for the first entering of a source section -> different types for a frame slot are ignored
- the snapshot taken copies frame and descriptor, but no deep copy of objects is done. Therefore the reference to these objects will represent the last state of execution

5.2. Limitations of Our Approach

- LSP: Syntax highlighting defined at client-side
- LSP+VSCode: Need to define supported languages in extension up-front
- LSP: No incremental sending of (completion) results
- LSP: has file-based code focus (GraalSqueak integration really hard)
- Run-time type collection / type harvesting
  - framework needs to support instrumentation / tracing to find the desired run-time object
  - needs possibility to query the resulting host language object about its guest language semantics (e.g. a TruffleObject understanding a KEYS message)
  - we query the actual run-time object, which might be really specialized, whereas type harvesting builds generalizations on seen types and yields only the generalized type

6. Related Work

6.1. Reusing existing tools

Helvetia [11] -> Helvetia has no performance focus, Graal-LSPperformance -> Helvetia DSL focus, Graal-LSPgeneral purpose focus, but DSLs possible -> Helvetia allows composing DSLs in same source, Graal-LSPhas Polyglot support, but only string based -> DSLs can use Smalltalk environment tools, but none else -> Graal-LSPlet's you choose the lsp client -> Helvetia translates dsls to Smalltalk by adding hooks into the Smalltalk compiler chain, Graal-LSP, i.e. Truffle uses language-specific interpreters -> Helvetia DSLs: syntax highlighting, debugger, -> but code completion, goto-definition etc only at Smalltalk level

Ensō is also mentioned in Language Workbenches

Self

Smalltalk+Squeak

Squimera [10] [9]

6.2. Language Workbenches

[2] -> not performance focused -> typically only one or a very limited set of target IDEs supported (e.g. an Eclipse plugin) -> not limited to textual notation (also graphical, tabular etc.) -> Composability -> Testing / debugging -> Editing mode: free form + projectional -> Syntactic services (Highlighting, Outline, Folding, Syntactic completion, Diff, Auto formatting) -> Semantic services (Reference resolution, Semantic completion, Refactoring, Error marking, Quick fixes, Origin tracking, Live Translation)
- stefan marr:

- Xtext, part of the Eclipse project, is a framework for defining external DSLs in close cooperation with the Eclipse IDE. [6]
- [15] use Spoofax and DynSem [14] -> generated AST-interpreter for Truffle <- can one modify the generated interpreter? -> end-user IDE: bound to the Spoofax generated editor(-plugin)
- Ensō http://enso-lang.org -> Ensō is a self-describing workbench based entirely on interpreters implemented in an object-oriented language. Ensō avoids the generation of code and follows the hypothesizes, that interpreters are easier to

develop and combine than compilers or transformational approaches. Because Ensō is implemented in itself, all available tools in the system can be reused to create applications. -> Smalltalk approach

6.3. Run-time type collection

- [5]
- Nemo Bachelor Project? -> sandboxed execution in Squeak
- https://github.com/dropbox/pyannotate -> Insert annotations into your source code based on call arguments and return types observed at runtime. - uses Python's sys.setprofile(profilefunc)
- https://blog.jetbrains.com/pycharm/2013/02/dynamic-runtime-type-inference-in-pycharm-2-7/

6.4. Language-agnostic tagging of symbols

Kythe

- A pluggable, (mostly) language-agnostic ecosystem for building tools that work with code.
- https://kythe.io/

-> LSP more specific, focus on IDE-like tooling missing in Truffle -> Kythe isn't really intended to be (or to replace) a UI-oriented tool like an editor or an IDE (LSP neither) -> Since Kythe analysis is static, it requires code that builds. Completion, by design, is intended for code that is in-progress, and generally doesn't (yet) compile or run. One could certainly use data from a Kythe index to augment local completion results, but the Kythe data won't solve the local problem of generating symbols for completion. -> but we explicitly want to support code completion

CTags

- tagging a language object in an index file
- tag-file supported by editors
- allows user to locate the tagged object via name (goto-definition, list symbols)
- for new languages you need to define a regex or parser to find language objects
- only static analysis, no scoped completion (everything is suggested)

SemanticDB

7. Conclusion and Future Work

Unique features

- Polyglot object property completion (e.g. suggest props of an JS object in python code)
- Goto-definition for functions which are first-class citizens

Will language implementers add needed interface impls to support most LSP features? We think yes, because you gain a lot with small changes and most of them are also needed for the Debugger (Expression Tags etc) or other tools

discuss this in Experience Report

7.1. Future Work

- Dynamic type collection
  - sandboxing -> copy-on-write FS
  - how to handle network I/O
  - smart caching -> store also frames which have different var types in a frame slot
- LSP features
  - Cancellation support for requests (In Truffle: ExecutionListener which checks each x statements if time limit exceeded -> throw Time-out-ThreadDeath-like exception)
  - Refactoring features
  - If semantic coloring is supported -> use that to compensate client-side syntax highlighting definition
- Truffle
  - how to enrich ASTs with parse-time infos without influencing run-time performance

Bibliography

[1] Muhammad Asaduzzaman. "Context-Sensitive Code Completion". PhD thesis. University of Saskatchewan, 2018.

[2] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The State of the Art in Language Workbenches". In: *International Conference on Software Language Engineering*. Springer. Springer International Publishing, 2013, pp. 197–217. DOI: 10.1007/978-3-319-02654-1_11.

[3] Yoshihiko Futamura. "Partial evaluation of computation process–an approach to a compiler-compiler". In: *Higher-Order and Symbolic Computation* 12.4 (1999), pp. 381–391. DOI: 10.1023/A:1010095604496.

[4] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. "High-performance cross-language interoperability in a multi-language runtime". In: *ACM SIGPLAN Notices*. Vol. 51. 2. ACM. 2015, pp. 78–90. DOI: 10.1145/2936313.2816714.

[5] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. "Type harvesting: a practical approach to obtaining typing information in dynamic programming languages". In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM. 2011, pp. 1282–1289. DOI: 10.1145/1982185.1982464.

[6] Christian Humer, Christian Wimmer, Christian Wirth, Andreas Wöß, and Thomas Würthinger. "A domain-specific language for building self-optimizing AST interpreters". In: *ACM SIGPLAN Notices* 50.3 (2015), pp. 123–132. DOI: 10.1145/2658761.2658776.

[7] Eva Maia, Nelma Moreira, and Rogério Reis. "A static type inference for python". In: *Proc. of DYLA* (2012).

[8] Microsoft. *Language Server Protocol*. https://microsoft.github.io/language-server-protocol/. Visited on 2018-09-24. 2017.

[9] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27. DOI: 10.1145/3079368.3079402.

[10] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* 2.3 (Mar. 2018). DOI: 10.22152/programming-journal.org/2018/2/8.

[11] Lukas Renggli, Tudor Gîrba, and Oscar Nierstrasz. "Embedding languages without breaking tools". In: *European Conference on Object-Oriented Programming*. Springer. 2010, pp. 380–404. DOI: 10.1007/978-3-642-14107-2_19.

[12] Michael L Van De Vanter. "Building debuggers and other tools: we can have it all". In: *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM. 2015, p. 2. DOI: 10.1145/2843915.2843917.

[13] Michael Van De Vanter, Chris Seaton, Michael Haupt, Christian Humer, and Thomas Würthinger. "Fast, Flexible, Polyglot Instrumentation Support for Debuggers and other Tools". In: *arXiv preprint arXiv:1803.10201* (2018). DOI: 10.22152/programming-journal.org/2018/2/14.

[14] Vlad Vergu, Pierre Neron, and Eelco Visser. "DynSem: A DSL for Dynamic Semantics Specification". In: *26th International Conference on Rewriting Techniques and Applications (RTA 2015)*. Ed. by Maribel Fernández. Vol. 36. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, pp. 365–378. ISBN: 978-3-939897-85-9. DOI: 10.4230/LIPIcs.RTA.2015.365.

[15] Eelco Visser, Guido Wachsmuth, Andrew Tolmach, Pierre Neron, Vlad Vergu, Augusto Passalaqua, and Gabrieël Konat. "A Language Designer's Workbench: A One-Stop-Shop for Implementation and Verification of Language Designs". In: *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM. 2014, pp. 95–111. DOI: 10.1145/2661136.2661149.

[16] Christian Wimmer, Michael Haupt, Michael L Van De Vanter, Mick Jordan, Laurent Daynès, and Douglas Simon. "Maxine: An approachable virtual machine for, and in, java". In: *ACM Transactions on Architecture and Code Optimization (TACO)* 9.4 (2013), p. 30. DOI: 10.1145/2400682.2400689.

[17] Christian Wimmer and Thomas Würthinger. "Truffle: a self-optimizing runtime system". In: *Proceedings of the 3rd annual conference on Systems, programming, and applications: software for humanity*. ACM. 2012, pp. 13–14. DOI: 10.1145/2384716.2384723.

[18] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. "Practical Partial Evaluation for High-performance Dynamic Language Runtimes". In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2017*. Vol. 52. 6. ACM. 2017, pp. 662–676. DOI: 10.1145/3062341.3062381.

[19] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. "One VM to rule them all". In: *Proceedings of the 2013 ACM international symposium on New ideas, new paradigms, and reflections on programming & software*. ACM. 2013, pp. 187–204. DOI: 10.1145/2509578.2509581.

[20] Thomas Würthinger, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Doug Simon, and Christian Wimmer. "Self-optimizing AST interpreters". In: *ACM SIGPLAN Notices*. Vol. 48. 2. ACM. 2012, pp. 73–82. DOI: 10.1145/2384577.2384587.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 26. Dezember 2018

Daniel Stolpe

Todo list

- explain deeper in extra sub-section? . . . 2
- footnote? . . . 5
- footnote? . . . 5
- ref . . . 6
- Tim fragen - werden ja nicht bei locals vorgeschlagen . . . 6
- figure: context instead of position, dots between completion and definition, dots at end . . . 7
- Is this important? . . . 8
- consume in a "Virtual Execution Environment" section? . . . 8
- ref Frame: "Advanced Compiler Design and Implementation" book p.114 . . . 9
- ref to Fowler, Spoofax, Xtext etc - footnote vs ref? . . . 9
- ref . . . 9
- cite or footnote? . . . 10
- bullet points to text . . . 11
- Überleitung . . . 12
- ref . . . 13
- far from perfect, difficult to built in a language-agnostic way, ... 13
- rewrite . . . 13
- explain in background? . . . 14
- ist das das richtige Wort für console I/O . . . 14
- ? . . . 14
- focus? . . . 16
- paragraph feels like to much implementation?! . . . 18
- label counter wrrong? . . . 20
- ref . . . 22
- is embedding-API the right term? . . . 22
- explain in background? . . . 23
- list problems? context-depedent code might be evaluated in global scope yielding wrong results etc. . . . 26
- discuss this in Experience Report . . . 41